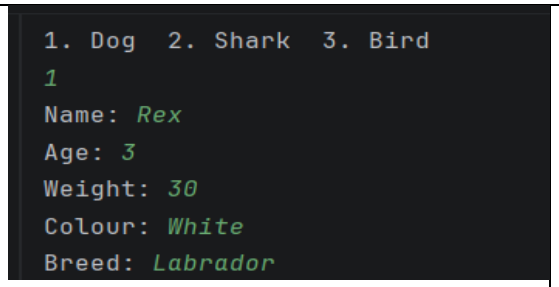
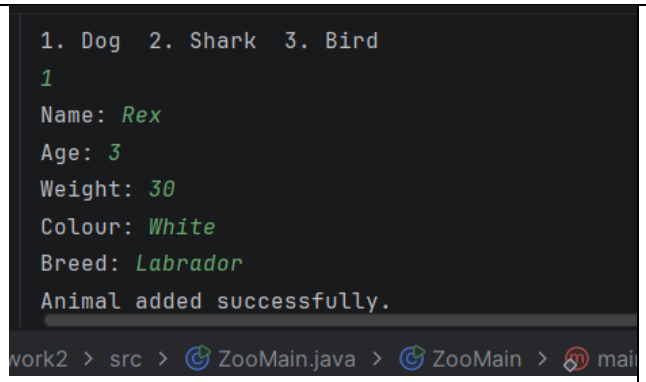
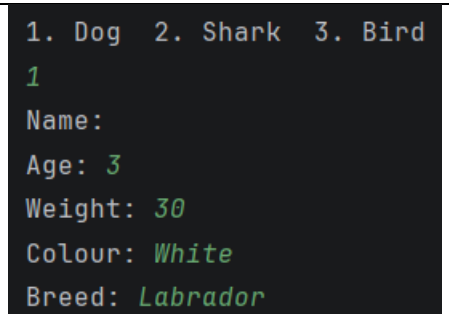
Testing

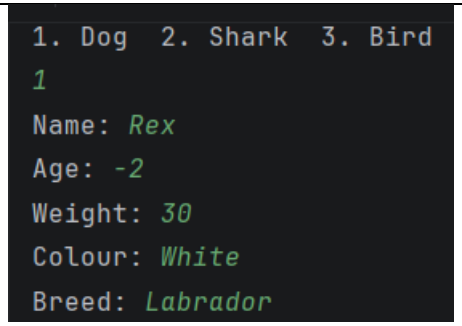
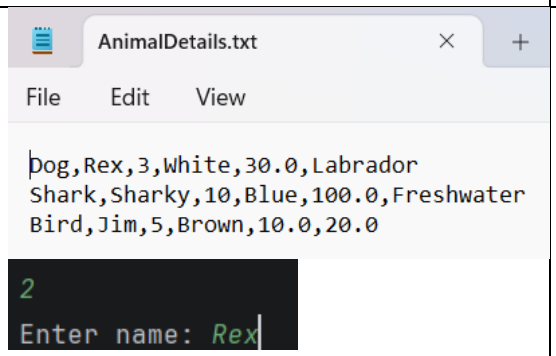
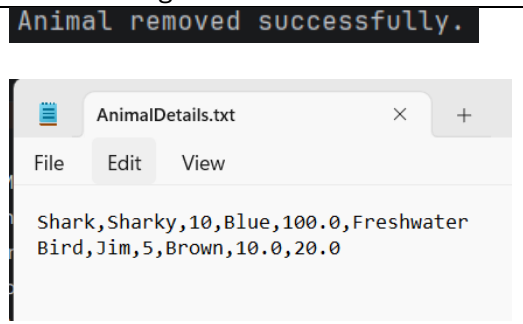
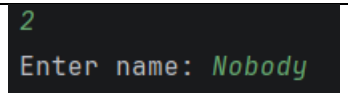
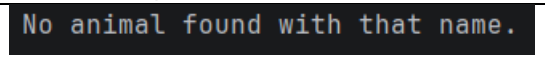
Testing Strategy

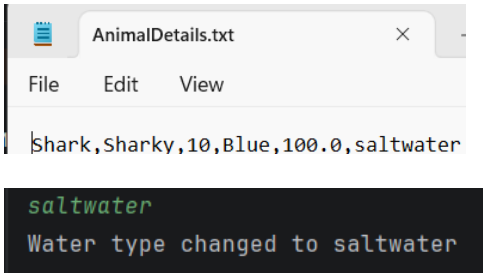
I focused my testing strategy on the key functional requirements of the coursework, focusing on testing functions such as the add, remove, edit, search, and view animals functions. For each function, I tested both valid and invalid inputs to confirm that the program handled errors appropriately, such as rejecting animals with missing or incorrect data using the isValid() method. I also tested invalid entries, such as searching for an animal that does not exist, editing values to invalid inputs, and removing animals that are not in the system. I also I verified that file handling worked correctly by saving data to files and then reloading it to ensure consistency. Finally, I tested the behaviour of subclasses and interfaces, ensuring that methods like makeSound(), fly(), and swim() worked correctly depending on the type of animal, demonstrating correct use of polymorphism.

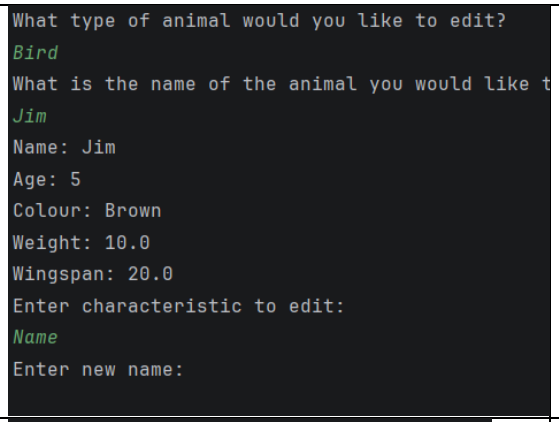
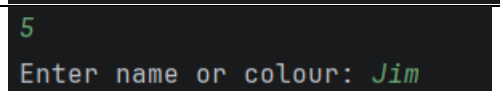
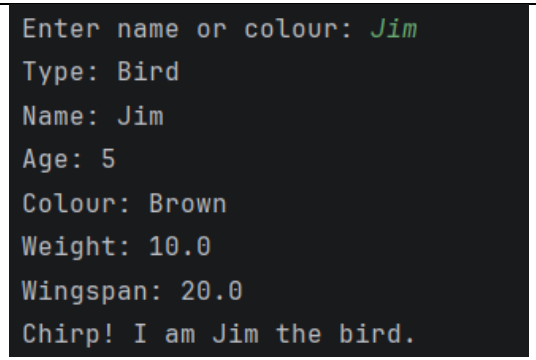
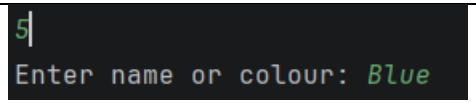
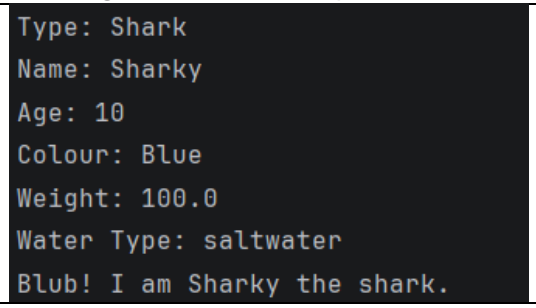
Test Case Table

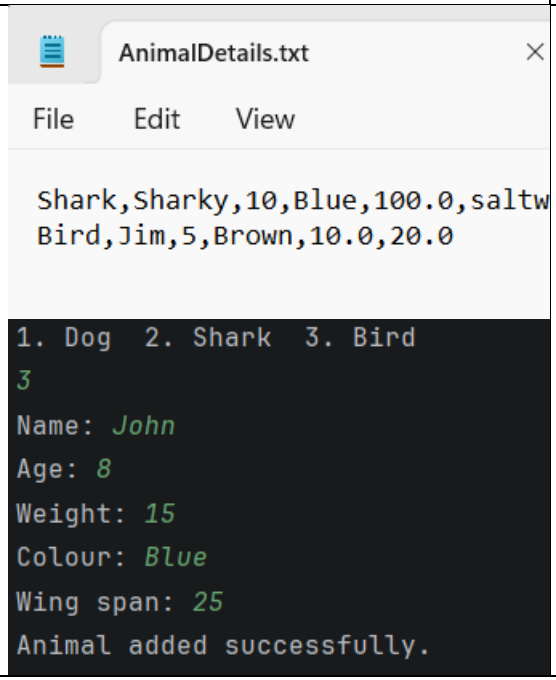
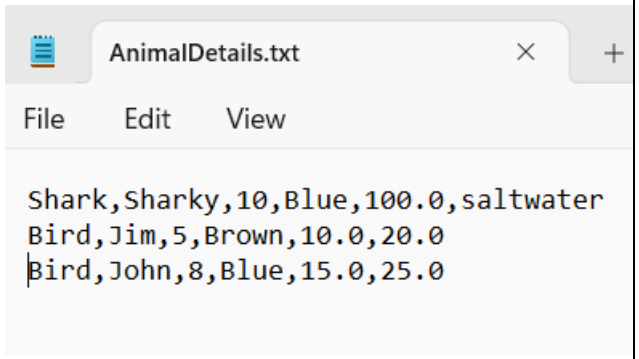
Test Case No.	**Test Case Description**	**Test Data**	**Expected Results**	**Actual Results**	**Status (Pass or fail)**

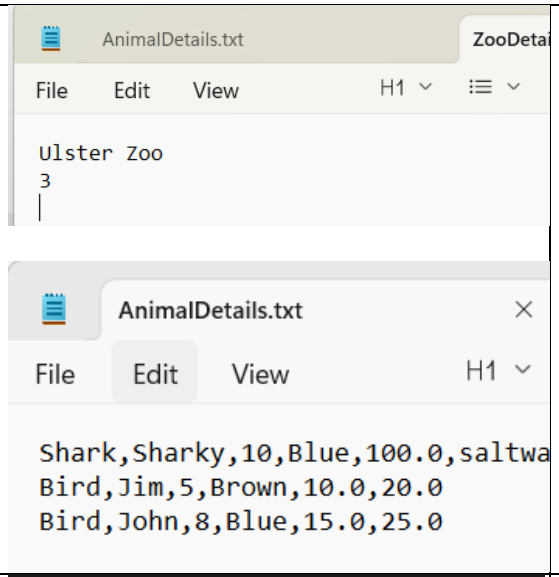
TC-01	Test of adding animal		Animal should add successfully and inform the user the animal has been added successfully	 The printed line shows the animal added successfully and all details are valid, means	P
TC-02	Test of adding an invalid animal (empty field)		The animal should not be saved and a message should show informing the user that the animal was invalid, meaning it also has not been added to the animal file.	 The blanked out name was considered invalid and has confirmed to the user that it was an invalid entry and the data has not saved	P

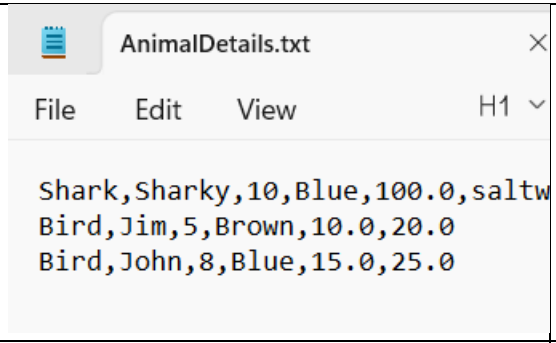
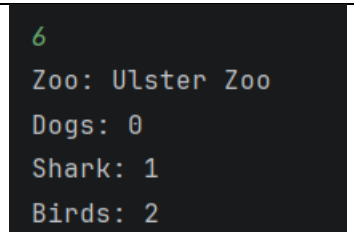
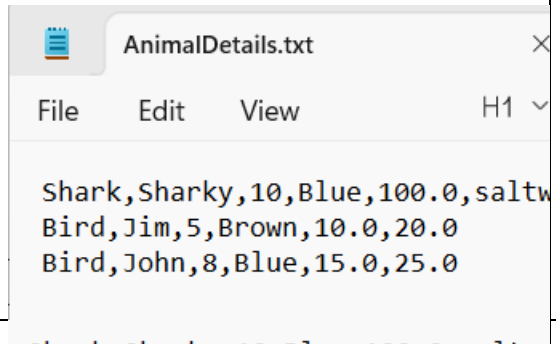
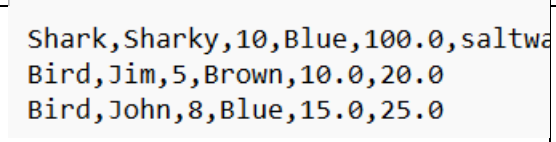
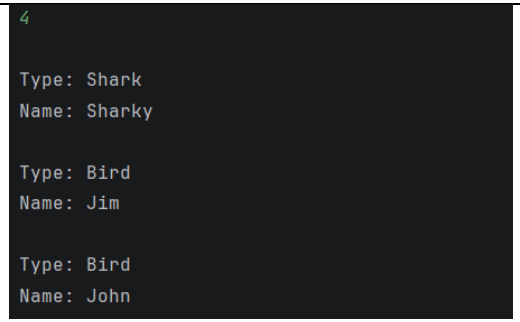
TC-03	Test of adding an invalid animal (negative age)		The animal should not be saved and a message should show informing the user that the animal was invalid, meaning it also has not been added to the animal file.	 The entry wasn't allowed to be saved as the age was a negative number so this prevents invalid entries being saved from invalid entries	P
TC-04	Removing an existing animal		Removing the animal should be shown in the file. Rex should no longer be present when the program is saved and closed and the file is opened	 The dog "Rex" was removed from the file showing that the remove has worked successfully	P
TC-05	Removing a non-existent animal		Warning message should be displayed informing the user that no animal could be found to remove	 No information came up and the user was informed that there was no animal found which shows this test has passed	P

TC-06	Edit a field with valid data	3 What type of animal would you like to edit? Shark What is the name of the animal you would like to e Sharky Name: Sharky Age: 10 Colour: Blue Weight: 100.0 Water type: Freshwater Enter characteristic to edit: water type Enter new water type: saltwater	Watertype should update to saltwater and when exiting the program, this should be updated in the file	Water type was updated to saltwater which was the expected result based on the input 	P
TC-07	Edit a field with invalid data	3 What type of animal would you like to edit? Bird What is the name of the animal you would like t Jim Name: Jim Age: 5 Colour: Brown Weight: 10.0 Wingspan: 20.0 Enter characteristic to edit: Weight Enter new weight: -10	The change should not be allowed and the user should get a message telling them that their update is invalid and can't be made	Invalid update. One or more fields invalid. Changes The update was not allowed to happen as there was an invalid field (weight) so this means no invalid entries can be entered.	P

TC-08	Edit a field with blank data		The change should not be allowed and the user should get a message telling them that their update is invalid and can't be made	 This update was not allowed to happen as the name was left blank ensuring no blank fields can be entered so the test has passed.	P
TC-09	Search by name		The bird called Jim should be loaded with his details	 Jim's details were loaded in successfully showing that this test has passed	P
TC-10	Search by colour		The shark called Sharky should be loaded with their details		P

				The blue animal “Sharky” has loaded in correctly from the search so this test has passed	
TC-11	Searching for a non-existing animal	5 Enter name or colour: Nobody	The user should be told the animal wasn’t found and no details should appear	Enter name or colour: Nobody Animal not found! The search didn’t produce a result and informed the user that no animal was found so the test has passed	P
TC-12	Save animals on exit		The added animal should be in the file when the file is opened after exiting the program	8. Save & Exit 8 Animals saved to file. Program ending.  The file has successfully been added to wit the new bird called John. This test has passed	P

TC-13	Data loads in from file		The zoo name should load in on the report	6 Zoo: Ulster Zoo Dogs: 0 Shark: 1 Birds: 2 Dominant colour: Blue The zoo name loaded in successfully from ZooDetails showing that this test was successful	P
TC-14	Flyable animal gets wing check	Name of animal for check: Jim	A message should be displayed to the user saying that the check for the day is complete	Name of animal for check: Jim Performing check for Jim Jim wings checked for today. The wing check proceeded it and it used the correct interface as Jim is one of the birds so this test has passed	P
TC-15	Swimmable animal gets water checked	Name of animal for check: sharky	A message should be displayed to the user saying that the check for the day is complete	Name of animal for check: sharky Performing check for Sharky Sharky water health checked for today. The check has been performed and has used the correct interface so this test has passed	P

TC-16	Report Count of animals is correct		The report should reflect what is seen in the file	 The animal count is correct based on the file so this test has passed	P
TC-17	Report of dominant colour is correct	From AnimalDetails.txt, there is 2 blue animals and 1 brown animal 	The dominant colour should be blue as there is 2 blue animals, 1 brown and none of any other colour	 The dominant colour has appeared as blue so this test has passed as there is 2 blue animals	P
TC-18	All animals are listed when animals are viewed		The animals should appear in the list with their type and name.	 The list was displayed with the type and name so this test has passed.	P

Unit Testing

I have unit tested my subclasses to verify the behaviour of individual subclasses of Animal. The methods tested included makeSound() to ensure correct output for each animal type, isValid() to validate input data, and getDetails() to confirm correct formatting for file storage. The expected output was that valid animals return true for validation and produce correct strings, while invalid animals return false.

Testing Dog Class:



```
    @Test
    public void testGetDetails() {
        Dog d = new Dog( name: "Buddy",  age: 3,  colour: "Brown",  weight: 10.
        String details = d.getDetails();

        assertTrue(details.contains("Dog"));
        assertTrue(details.contains("Buddy"));
        assertTrue(details.contains("Labrador"));
    }
}
```

```
public class DogTest {
    @Test
    public void testMakeSound() {
        Dog d = new Dog( name: "Buddy",  age: 3,  colour: "Brown",  weight: 10.
        String result = d.makeSound();

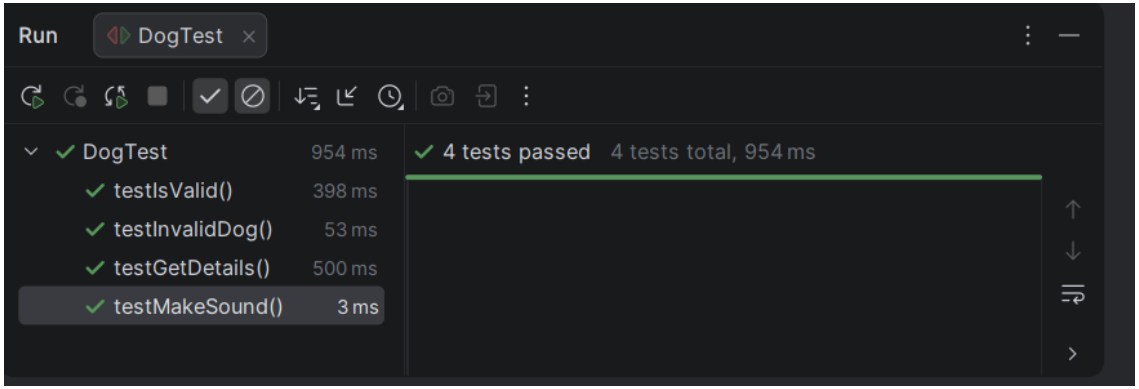
        assertTrue(result.contains("Woof"));
        assertTrue(result.contains("Buddy"));
    }

    @Test
    public void testIsValid() {
        Dog d = new Dog( name: "Buddy",  age: 3,  colour: "Brown",  weight: 10.
        assertTrue(d.isValid());
    }

    @Test
    public void testInvalidDog() {
        Dog d = new Dog( name: "",  age: -1,  colour: "",  weight: -5,  breed: "
        assertFalse(d.isValid());
```

Results:

All tests in the dog class were passed.



Testing Shark Class:

```java
@Test
public void testMakeSound() {
    Shark s = new Shark( name: "Jaws", age: 5, colour: "Grey", weight: 200.0, waterType: "Saltwater");
    String result = s.makeSound();

    assertTrue(result.contains("Blub"));
    assertTrue(result.contains("Jaws"));
}

@Test
public void testIsValid() {
    Shark s = new Shark( name: "Jaws", age: 5, colour: "Grey", weight: 200.0, waterType: "Saltwater");
    assertTrue(s.isValid());
}

@Test
public void testInvalidShark() {
    Shark s = new Shark( name: "", age: -1, colour: "", weight: -10.0, waterType: "Saltwater");
    assertFalse(s.isValid());
}
```

```java
@Test
public void testGetDetails() {
    Shark s = new Shark( name: "Jaws",  age: 5,  colour: "Grey",  weight: 200.0,  waterType: "Saltwater");
    String details = s.getDetails();

    assertTrue(details.contains("Shark"));
    assertTrue(details.contains("Jaws"));
    assertTrue(details.contains("Saltwater"));
}

@Test
public void testSwimMethod() {
    Shark s = new Shark( name: "Jaws",  age: 5,  colour: "Grey",  weight: 200.0,  waterType: "Saltwater");

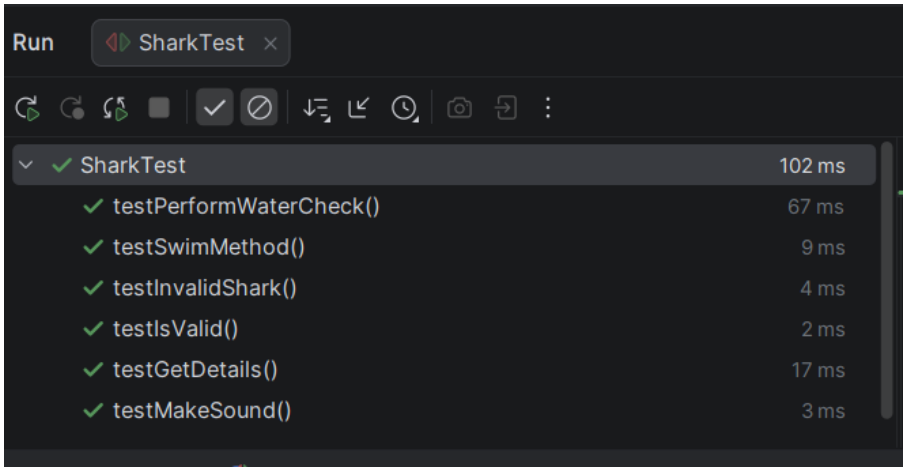
    // Just ensure it runs without crashing
    assertDoesNotThrow(() -> s.swim());
}

@Test
public void testPerformWaterCheck() {
    Shark s = new Shark( name: "Jaws",  age: 5,  colour: "Grey",  weight: 200.0,  waterType: "Saltwater");

    // Just ensure it runs without crashing
    assertDoesNotThrow(() -> s.performWaterCheck());
}}
```

Results:

All tests in the shark class passed



Testing bird class:

```java
public class BirdTest {

    @Test
    public void testMakeSound() {
        Bird b = new Bird( name: "Tweety",  age: 2,  colour: "Yellow",  weight: 1.5,  wingSpan: 0.3);
        String result = b.makeSound();

        assertTrue(result.contains("Chirp"));
        assertTrue(result.contains("Tweety"));
    }

    @Test
    public void testIsValid() {
        Bird b = new Bird( name: "Tweety",  age: 2,  colour: "Yellow",  weight: 1.5,  wingSpan: 0.3);
        assertTrue(b.isValid());
    }

    @Test
    public void testInvalidBird() {
        Bird b = new Bird( name: "",  age: -1,  colour: "",  weight: -2.0,  wingSpan: 0.3);
        assertFalse(b.isValid());
    }
```

```java
    @Test
    public void testGetDetails() {
        Bird b = new Bird( name: "Tweety",  age: 2,  colour: "Yellow",  weight: 1.5,  wingSpan: 0.3);
        String details = b.getDetails();

        assertTrue(details.contains("Bird"));
        assertTrue(details.contains("Tweety"));
    }

    @Test
    public void testFlyMethod() {
        Bird b = new Bird( name: "Tweety",  age: 2,  colour: "Yellow",  weight: 1.5,  wingSpan: 0.3);

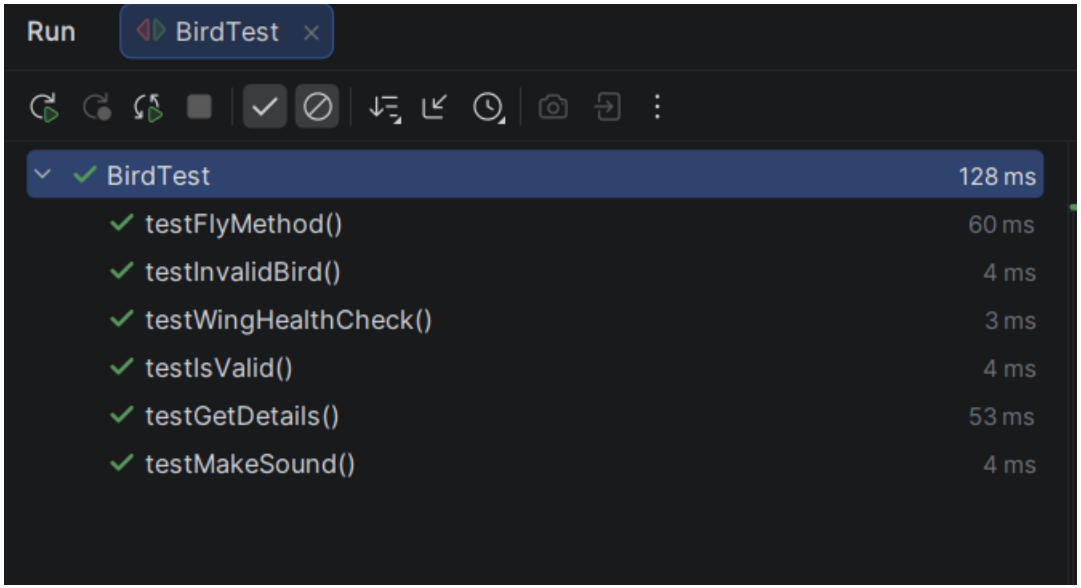
        // Ensure no exception occurs
        assertDoesNotThrow(() -> b.fly());
    }

    @Test
    public void testWingHealthCheck() {
        Bird b = new Bird( name: "Tweety",  age: 2,  colour: "Yellow",  weight: 1.5,  wingSpan: 0.3);

        // Ensure no exception occurs
        assertDoesNotThrow(() -> b.wingHealthCheck());
    }
}
```

Results:

All bird class tests passed



Requirements Table

Requirement	Implemented (Yes/No)	Test Case linked to Requirement

1. The application must be able to handle data from at least three different types of animals implementing at least one interface such as Swimmable and Flyable. Each interface must contain more than one abstract methods.	Yes	TC-14-15
2. For each animal, their name, age, colour, and weight should be recorded	Yes	TC-01-03
3. You must introduce at least one additional instance variable relevant to the specific type of animal for example, you could introduce a variable representing milkProduction for a cow.	Yes	TC-01 (Dog breed variable)
4. The program should contain a method called makeSound() that returns a typical animal noise, plus a description of the animal such as ..	Yes	TC-09-10
5. The program needs to enable animals to be added to the zoo or removed from the zoo. It should also have any facility to modify animal details after they have been created.	Yes	TC-01-08
6. The program should allow a zookeeper to perform daily care for all animals including specialized care based on the interfaces implemented for example performing a general wing health check for flyable animals.	Yes	TC-14-15
7. It must be possible to report (i.e. print to the screen) on the zoo. This report should indicate the zoo name which could be hard-coded, the total number of each type of animals	Yes	TC13+16-17

registered and the dominant color for these animals.		
8. The program should allow the user to view all the animals currently registered at the zoo.	Yes	TC-18
9. At the end of a session, when the program is being terminated, the zoo details along with the details for each registered animal should be written to disk. These details should be recorded in two standard text files: one containing zoo details (called “ZooDetails.txt”) and the other holding all the animal records, to be called “AnimalDetails.txt”.	Yes	TC-12
10. When the program is started, it should read from these standard text files to re-populate the application with any previously stored data as a starting point	Yes	TC-13
The program user must also be able to search for an animal by name or color, causing the sought animal’s details to be displayed and its makeSound( )method to be called if present	Yes	TC-09-11